

Primer Parcial Práctico

Cinemática Directa e Inversa del Robot Universal Robots UR10e con ROS 2 Jazzy

Grupo 6

Sebastian Samuel Ariñez Quispe

Marco Andres Jaldín Mendoza

IMT-342 Robótica – Universidad Católica Boliviana “San Pablo”

Docente: Carlos Daniel Aguilar Mancachi

Resumen—Este documento presenta el modelado y la implementación de la cinemática directa e inversa para el manipulador robótico Universal Robots UR10e de 6 grados de libertad, utilizando el ecosistema ROS 2 Jazzy. Se estableció el modelado matemático empleando la convención Denavit-Hartenberg (DH) estándar para la asignación de marcos de referencia y la deducción de los parámetros geométricos. Posteriormente, se desarrollaron las matrices de transformación homogénea individuales y la matriz final T_{06} , verificando la posición y orientación del efector final simbólicamente y numéricamente con errores menores que 10^{-6} m respecto del modelo oficial. Se dedujo el Jacobiano posicional de la estructura, demostrando la concordancia entre el método analítico y el geométrico, y se analizaron las restricciones articulares y singularidades mecánicas del modelo. Finalmente, se diseñó e implementó un método iterativo de cinemática inversa basado en la pseudoinversa del Jacobiano para alcanzar posiciones cartesianas deseadas mediante tópicos de ROS 2. Todo el sistema fue validado y simulado utilizando RViz2.

Index Terms—Denavit-Hartenberg, cinemática directa, cinemática inversa, Jacobiano, ROS 2, RViz2, manipulador serial, UR10e.

I. DESCRIPCIÓN DEL ROBOT ASIGNADO

El robot asignado para este proyecto es el manipulador colaborativo UR10e fabricado por Universal Robots. Este modelo destaca por su amplio espacio de trabajo y capacidad de carga, siendo ampliamente utilizado en entornos industriales para tareas de paletizado, ensamblaje y soldadura. Su estructura mecánica consiste en una cadena serial de seis eslabones interconectados por articulaciones completamente revolutas, proporcionando 6 grados de libertad (GDL).

I-A. Datos generales

A continuación, se detallan las especificaciones técnicas y geométricas fundamentales extraídas de la hoja técnica oficial del fabricante y del modelo en ROS 2. [1]

Cuadro I
DATOS PRINCIPALES DEL ROBOT UR10E

Dato	Valor
Fabricante	Universal Robots
Modelo	UR10e
Grados de libertad	6 (Articulaciones revolutas)
Alcance máximo	1300 mm
Carga útil	12.5 kg
Marco base	base (DH); base_link está rotado $R_z(\pi)$
Efector final	tool0
Unidades empleadas	SI: metros y radianes

I-B. Cadena cinemática

La estructura del UR10e se caracteriza por una disposición particular en la que los ejes de las articulaciones del hombro, codo y la primera muñeca (articulaciones 2, 3 y 4) son paralelos entre sí. Esta geometría simplifica el desarrollo matemático, pero concentra los desplazamientos laterales en parámetros específicos que se detallarán en la asignación de marcos. En la Figura 1 se identifican las 6 articulaciones $q_1 \dots q_6$ utilizadas para controlar la posición de la brida final.

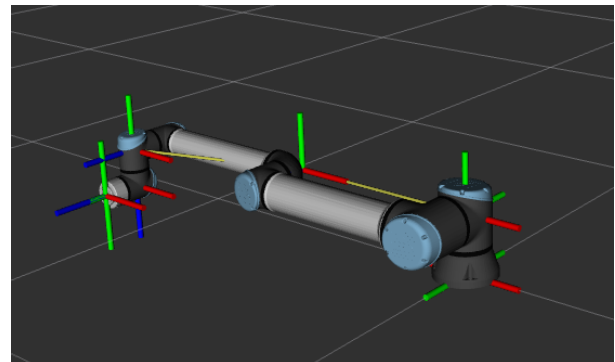


Figura 1. Robot UR10e y articulaciones consideradas (Captura de RViz2).

II. ASIGNACIÓN DE MARCOS DE REFERENCIA

Para establecer la cinemática del UR10e, se ubicaron los marcos de referencia en cada articulación siguiendo reglas geométricas estrictas para manipuladores seriales. Para cada marco i (del 1 al 6):

- **Origen O_i :** Se ubicó en la intersección de la normal común y el eje de la articulación correspondiente.
- **Eje z_i :** Alineado de manera coincidente con el eje de rotación de cada motor revoluto.
- **Eje x_i :** Orientado a lo largo de la normal común entre z_{i-1} y z_i . Cuando los ejes son paralelos (articulaciones 2, 3 y 4) la normal común no es única; se eligió el sentido de x_i opuesto al avance del brazo, igual que el eje x del marco base de ROS. Con esta elección todos los x_i son paralelos en $q = 0$, los offsets de θ_i son nulos y a_2, a_3 resultan negativos.
- **Eje y_i :** Definido por la regla de la mano derecha para completar el sistema ortogonal cartesiano.

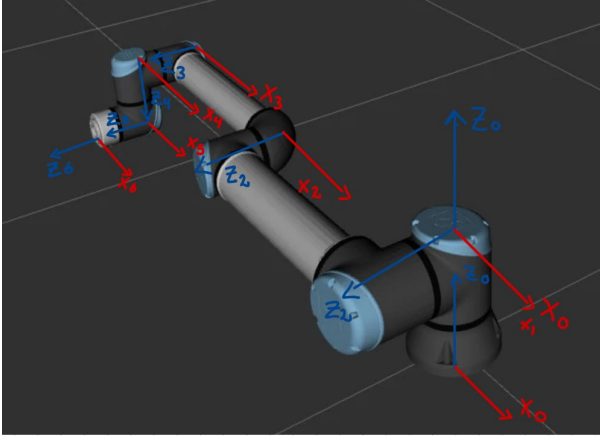


Figura 2. Asignación de marcos de referencia del robot UR10e.

II-A. Convención utilizada

En este trabajo se utilizará la convención Denavit–Hartenberg estándar: [4]

$$A_{i-1}^i = R_z(\theta_i) T_z(d_i) T_x(a_i) R_x(\alpha_i). \quad (1)$$

Su forma matricial es:

$$A_{i-1}^i = \begin{bmatrix} c_{\theta_i} & -s_{\theta_i} c_{\alpha_i} & s_{\theta_i} s_{\alpha_i} & a_i c_{\theta_i} \\ s_{\theta_i} & c_{\theta_i} c_{\alpha_i} & -c_{\theta_i} s_{\alpha_i} & a_i s_{\theta_i} \\ 0 & s_{\alpha_i} & c_{\alpha_i} & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}. \quad (2)$$

donde $c_{\theta_i} = \cos \theta_i$ y $s_{\theta_i} = \sin \theta_i$.

III. PARÁMETROS DENAVIT–HARTENBERG

Basado en la asignación de marcos de referencia y extrayendo las dimensiones físicas del modelo URDF oficial, se construyó la Tabla II. Cabe destacar que se implementó una disposición estándar, lo que elimina la necesidad de añadir offsets artificiales dentro de los ángulos articulares θ_i . [2], [3]

III-A. Verificación de los parámetros con el URDF oficial

Los valores de la Tabla II coinciden con los parámetros DH publicados por Universal Robots para el UR10e [2]. Para contrastarlos con el modelo usado en ROS 2 se revisó el

paquete `ur_description` [3]. En ese paquete, la geometría cinemática no está en `physical_parameters.yaml`, que contiene masas, inercias y los desplazamientos de las mallas, sino en `config/ur10e/default_kinematics.yaml`. Este archivo define el origen (`xyz, rpy`) de cada `joint`, y `ur_macro.xacro` lo usa para construir el URDF. La Tabla III muestra la correspondencia uno a uno.

Cuadro III
PARÁMETROS DH FRENTE AL URDF OFICIAL
(`DEFAULT_KINEMATICS.YAML`)

DH	Valor	Joint del URDF	Campo del origen
d_1	0.1807	shoulder_pan	$z = 0.1807$
α_1	$\pi/2$	shoulder_lift	$\text{roll} = \pi/2$
a_2	-0.6127	elbow	$x = -0.6127$
a_3	-0.57155	wrist_1	$x = -0.57155$
d_4	0.17415	wrist_1	$z = 0.17415$
d_5	0.11985	wrist_2	$y = -0.11985$
d_6	0.11655	wrist_3	$y = 0.11655$

Todas las magnitudes coinciden exactamente; solo cambian el eje o el signo con que aparecen, porque los marcos de los eslabones del URDF no tienen la misma orientación que los marcos DH. Por ejemplo, d_5 aparece como $y = -0.11985$ porque el eje z_4 del marco DH corresponde al eje $-y$ del marco de `wrist_1_link`. Su interpretación geométrica es la siguiente:

- $d_1 = 0.1807$ m: altura del eje del hombro sobre la base, a lo largo de z_0 .
- $a_2 = -0.6127$ m y $a_3 = -0.57155$ m: longitudes del brazo y del antebrazo. Son negativas porque x_2 y x_3 apuntan en sentido opuesto al avance del brazo.
- $d_4 = 0.17415$ m: desplazamiento lateral total entre el plano del brazo y la muñeca 1, medido a lo largo de z_3 . Como los ejes 2, 3 y 4 son paralelos, este desplazamiento puede concentrarse en un solo parámetro ($d_2 = d_3 = 0$). El URDF hace lo mismo: lo coloca completo en el origen de `wrist_1_joint`. Los valores `shoulder_offset` y `elbow_offset` de `physical_parameters.yaml` solo desplazan las mallas visuales y no afectan la cinemática.
- $d_5 = 0.11985$ m: distancia entre los ejes de la muñeca 1 y la muñeca 3, a lo largo del eje de la muñeca 2 (z_4).
- $d_6 = 0.11655$ m: distancia desde el eje de la muñeca 2 hasta la brida (`tool0`), a lo largo del eje de la muñeca 3 (z_5).

Además, en $q = 0$ la configuración del URDF coincide con la de la tabla DH. Por eso no se requieren offsets articulares ($\theta_i = q_i$), y los ángulos publicados en `/joint_states` se usan directamente en las matrices DH. Esta correspondencia se confirma numéricamente en la Sección ??, donde T_0^6 reproduce la transformación `base→tool0` de TF con error menor que 10^{-6} m.

El UR10e no tiene muñeca esférica: los ejes de las articulaciones 4, 5 y 6 no se cortan en un punto común ($d_5 \neq 0$). Esto impide el desacople cinemático clásico y justifica el uso de una cinemática inversa numérica.

El marco base DH coincide con el marco base del URDF. Este marco está rotado π

Cuadro II
PARÁMETROS DENAVIT–HARTENBERG DEL ROBOT UR10E (ESTÁNDAR)

Transformación	θ_i	d_i [m]	a_i [m]	α_i [rad]	Descripción
A_0^1	q_1	0.1807	0	$\pi/2$	base $\rightarrow$ hombro
A_1^2	q_2	0	-0.6127	0	hombro $\rightarrow$ codo
A_2^3	q_3	0	-0.57155	0	codo $\rightarrow$ muñeca 1
A_3^4	q_4	0.17415	0	$\pi/2$	muñeca 1 $\rightarrow$ muñeca 2
A_4^5	q_5	0.11985	0	$-\pi/2$	muñeca 2 $\rightarrow$ muñeca 3
A_5^6	q_6	0.11655	0	0	muñeca 3 $\rightarrow$ tool0

alrededor de z respecto de `base_link` (joint fijo `base_link-base_fixed_joint`), porque `base_link` sigue la convención REP-103 (x hacia adelante), mientras que el controlador de UR usa x hacia atrás. Esta transformación fija no forma parte de la tabla DH y se expresa por separado:

$$T_{\text{base_link}}^{\text{base}} = \begin{bmatrix} -1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}. \quad (3)$$

Todas las comparaciones con ROS 2 se realizan respecto de `base`.

IV. CINEMÁTICA DIRECTA

La cinemática directa se obtiene mediante el producto secuencial de las seis matrices de transformación homogénea de la Tabla II, construidas con la ecuación (2):

$$T_0^6 = A_0^1 A_1^2 A_2^3 A_3^4 A_4^5 A_5^6 = \begin{bmatrix} R_0^6 & p(q) \\ \mathbf{0}^T & 1 \end{bmatrix}, \quad (4)$$

donde $R_0^6 \in SO(3)$ es la orientación de `tool0` y $p(q) \in \mathbb{R}^3$ su posición, ambas expresadas en el marco `base`. Si se requiere la pose respecto de `base_link`, basta premultiplicar por la transformación fija definida junto a la tabla DH: $T_{\text{base_link}}^{\text{tool0}} = T_{\text{base_link}}^{\text{base}} T_0^6$.

En lo que sigue se usa la notación $c_i = \cos q_i$, $s_i = \sin q_i$, $c_{23} = \cos(q_2 + q_3)$, $s_{23} = \sin(q_2 + q_3)$, $c_{234} = \cos(q_2 + q_3 + q_4)$ y $s_{234} = \sin(q_2 + q_3 + q_4)$.

IV-A. Matrices homogéneas individuales

Sustituyendo cada fila de la Tabla II en la ecuación (2) ($\theta_i = q_i$, sin offsets):

$$\begin{aligned} A_0^1 &= \begin{bmatrix} c_1 & 0 & s_1 & 0 \\ s_1 & 0 & -c_1 & 0 \\ 0 & 1 & 0 & d_1 \\ 0 & 0 & 0 & 1 \end{bmatrix}, & A_1^2 &= \begin{bmatrix} c_2 & -s_2 & 0 & a_2 c_2 \\ s_2 & c_2 & 0 & a_2 s_2 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \\ A_2^3 &= \begin{bmatrix} c_3 & -s_3 & 0 & a_3 c_3 \\ s_3 & c_3 & 0 & a_3 s_3 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, & A_3^4 &= \begin{bmatrix} c_4 & 0 & s_4 & 0 \\ s_4 & 0 & -c_4 & 0 \\ 0 & 1 & 0 & d_4 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \\ A_4^5 &= \begin{bmatrix} c_5 & 0 & -s_5 & 0 \\ s_5 & 0 & c_5 & 0 \\ 0 & -1 & 0 & d_5 \\ 0 & 0 & 0 & 1 \end{bmatrix}, & A_5^6 &= \begin{bmatrix} c_6 & -s_6 & 0 & 0 \\ s_6 & c_6 & 0 & 0 \\ 0 & 0 & 1 & d_6 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \end{aligned} \quad (5)$$

con $d_1 = 0.1807$, $a_2 = -0.6127$, $a_3 = -0.57155$, $d_4 = 0.17415$, $d_5 = 0.11985$ y $d_6 = 0.11655$ m. Las matrices A_1^2 , A_2^3 y A_5^6 son rotaciones puras alrededor de z ($\alpha_i = 0$), mientras que A_0^1 , A_3^4 y A_4^5 incluyen el giro de $\pm\pi/2$ que reorienta el eje de la articulación siguiente.

IV-B. Posición del efector final

Al multiplicar la cadena se definen dos términos auxiliares:

$$W = a_2 c_2 + a_3 c_{23} + d_5 s_{234} - d_6 s_5 c_{234}, \quad (6)$$

$$V = d_4 + d_6 c_5, \quad (7)$$

con los cuales la posición de `tool0` queda

$$p(q) = \begin{bmatrix} c_1 W + s_1 V \\ s_1 W - c_1 V \\ d_1 + a_2 s_2 + a_3 s_{23} - d_5 c_{234} - d_6 s_5 s_{234} \end{bmatrix}. \quad (8)$$

La estructura de (8) refleja la geometría del robot. Las sumas $q_2 + q_3$ y $q_2 + q_3 + q_4$ aparecen agrupadas porque los ejes 2, 3 y 4 son paralelos. W es la componente del efector en el plano del brazo, a lo largo de $x_1 = [c_1, s_1, 0]^T$, y V es su desplazamiento lateral a lo largo del eje del hombro $z_1 = [s_1, -c_1, 0]^T$; es decir, $p = W x_1 + V z_1 + p_z z_0$. Además, q_6 no aparece en $p(q)$, lo que anticipa la columna nula del Jacobiano posicional.

IV-C. Orientación del efector final

La matriz de rotación

$$R_0^6 = \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix} \quad (9)$$

tiene los términos

$$\begin{aligned} r_{11} &= (c_1 c_5 c_{234} + s_1 s_5) c_6 - c_1 s_{234} s_6 \\ r_{12} &= -(c_1 c_5 c_{234} + s_1 s_5) s_6 - c_1 s_{234} c_6 \\ r_{13} &= s_1 c_5 - c_1 s_5 c_{234} \\ r_{21} &= (s_1 c_5 c_{234} - c_1 s_5) c_6 - s_1 s_{234} s_6 \\ r_{22} &= -(s_1 c_5 c_{234} - c_1 s_5) s_6 - s_1 s_{234} c_6 \\ r_{23} &= -c_1 c_5 - s_1 s_5 c_{234} \\ r_{31} &= c_5 s_{234} c_6 + c_{234} s_6 \\ r_{32} &= -c_5 s_{234} s_6 + c_{234} c_6 \\ r_{33} &= -s_5 s_{234} \end{aligned} \quad (10)$$

La tercera columna $[r_{13}, r_{23}, r_{33}]^T$ es el eje z_6 de `tool0`, que coincide con z_5 y no depende de q_6 ; por eso $p(q)$ contiene

el término $d_6 z_5$ sin q_6 . Las expresiones (8) y (10) se obtuvieron con SymPy (`scripts/dh_sympy.py`) y coinciden término a término con el producto numérico de las matrices A implementado en `ur10e_dh.py`.

IV-D. Comprobaciones matemáticas

Como primera verificación, en $q = 0$ todos los senos se anulan y la cadena se reduce a

$$T_0^6(0) = \begin{bmatrix} 1 & 0 & 0 & a_2 + a_3 \\ 0 & 0 & -1 & -(d_4 + d_6) \\ 0 & 1 & 0 & d_1 - d_5 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad (11)$$

es decir, $p(0) = [-1.1843, -0.2907, 0.0609]^T$ m: el brazo completamente extendido en la dirección $-x$ de base, con la brida desplazada lateralmente $d_4 + d_6$ y una distancia d_5 por debajo de la altura del hombro.

En todas las configuraciones evaluadas la rotación calculada cumple $\|(R_0^6)^T R_0^6 - I\|_{\max} \approx 2 \times 10^{-16}$ y $\det(R_0^6) = 1$ al nivel del redondeo de punto flotante. La comparación cuantitativa contra la transformación `base→tool0` publicada por TF se presenta en la Sección IV-A (Tabla VI): con estados articulares publicados con precisión completa, el error es menor que 10^{-6} m en posición y 10^{-6} rad en orientación.

V. JACOBIANO DEL MANIPULADOR

Para el método iterativo de la cinemática inversa (IK) se requiere calcular el Jacobiano posicional $J_v(q)$ del efector final, el cual relaciona las velocidades articulares con la velocidad lineal cartesiana.

V-A. Jacobiano utilizado en el proyecto

El Jacobiano posicional fue deducido utilizando herramientas simbólicas mediante dos vías matemáticas para comprobar su validez: el método analítico (derivadas parciales de p_x, p_y, p_z respecto a $q_1 \dots q_6$) y el método geométrico, demostrando numéricamente una diferencia entre ambos métodos de apenas 1.6×10^{-17} .

La estructura general del Jacobiano 3×6 utilizado en el algoritmo es:

$$J_v(q) = \begin{bmatrix} \frac{\partial p_x}{\partial q_1} & \frac{\partial p_x}{\partial q_2} & \dots & \frac{\partial p_x}{\partial q_6} \\ \frac{\partial p_y}{\partial q_1} & \frac{\partial p_y}{\partial q_2} & \dots & \frac{\partial p_y}{\partial q_6} \\ \frac{\partial p_z}{\partial q_1} & \frac{\partial p_z}{\partial q_2} & \dots & \frac{\partial p_z}{\partial q_6} \end{bmatrix} \quad (12)$$

Cabe destacar que el cálculo de la sexta columna arroja consistentemente un vector nulo $[0 \ 0 \ 0]^T$. Esto se justifica físicamente porque el motor `wrist_3` (q_6) tiene la función exclusiva de rotar el efector sobre su propio eje Z_5 , alterando únicamente la orientación (R_0^6) pero sin aportar ningún desplazamiento al centro del marco de referencia en el espacio tridimensional (x, y, z) . [m]

V-B. Consideración de singularidades

Cuando J_v pierde rango, existen direcciones del espacio en las que el efector no puede moverse, y la pseudoinversa produce incrementos articulares muy grandes cerca de esas configuraciones. En el UR10e se distinguen tres singularidades típicas: de hombro (el efector sobre el eje z_0), de codo ($q_3 = 0$ o π , brazo estirado) y de muñeca ($q_5 = 0$ o π , ejes 4 y 6 paralelos).

Se evaluó J_v en $q = 0$ (brazo completamente extendido). Sus valores singulares son 1.33, 1.22 y 0.17, por lo que el rango es 3 y la matriz no es singular. Sin embargo, el par hombro-codo por sí solo sí está casi en singularidad: las columnas 2 y 3 tienen valores singulares 1.32 y 0.055. La dirección que pierde el codo la compensa la muñeca, porque `tool0` está desplazado d_5 respecto del eje de la muñeca 1. Como $q = 0$ además es el límite del espacio de trabajo, se usa como semilla una configuración con el codo doblado.

VI. CINEMÁTICA INVERSA

VI-A. Definición del problema

La cinemática inversa (IK) busca una configuración articular $q^* = [q_1 \dots q_6]^T$ tal que la posición del efector alcance un objetivo cartesiano

$$p_d = [x_d \ y_d \ z_d]^T, \quad p(q^*) \approx p_d. \quad (13)$$

En este trabajo la IK es solo de posición: la orientación del efector no se impone, aunque sí se calcula en la cinemática directa.

VI-B. Objetivo recibido mediante ROS 2

El objetivo se recibe en el tópico `/target` con mensajes `geometry_msgs/msg/Point`, expresado en metros respecto del marco base. Cada mensaje recibido inicia una nueva resolución. Por ejemplo:

```
ros2 topic pub --once /target geometry_msgs/msg/
Point "{x: 0.40, y: 0.20, z: 0.50}"
```

VI-C. Método numérico iterativo

Con el error de posición en la iteración k , $e_k = p_d - p(q_k)$, la actualización es

$$q_{k+1} = q_k + \alpha J_v(q_k)^+ e_k, \quad (14)$$

donde α es el factor de actualización y $J_v(q_k)^+$ es la pseudoinversa de Moore–Penrose del Jacobiano posicional geométrico. Es un método de tipo Newton–Raphson: linealiza $p(q)$ alrededor de q_k ($\Delta p \approx J_v \Delta q$) y resuelve ese sistema en el sentido de mínimos cuadrados. Como J_v es de 3×6 , el sistema tiene infinitas soluciones y J_v^+ entrega la de norma mínima. La pseudoinversa se calcula con `numpy.linalg.pinv`, que la obtiene mediante la descomposición en valores singulares. Para evitar pasos demasiado grandes (fuera de la zona donde la linealización es válida o cerca de singularidades), además de α se limita el cambio de cada articulación por iteración a `paso_max`: si algún $|\Delta q_i|$ lo supera, todo el vector Δq se escala proporcionalmente, conservando su dirección.

VI-D. Configuración inicial

El algoritmo parte siempre de la misma semilla

$$q_0 = [0, -\frac{\pi}{2}, \frac{\pi}{2}, -\frac{\pi}{2}, -\frac{\pi}{2}, 0] \text{ rad}, \quad (15)$$

una configuración con el codo doblado y la muñeca hacia abajo, alejada del límite del espacio de trabajo (se evita $q = 0$, que corresponde al brazo completamente estirado). Usar siempre la misma semilla hace que cada prueba sea reproducible.

VI-E. Articulaciones utilizadas

Las seis articulaciones participan en la actualización. Sin embargo, como la columna 6 de J_v es nula, la solución de norma mínima nunca modifica q_6 , que conserva el valor de la semilla. Esto es coherente con una IK de solo posición: `wrist_3` únicamente cambia la orientación del efector.

VI-F. Límites articulares

Los límites se tomaron de `config/ur10e/joint_limits.yaml` del paquete `ur_description` (Tabla IV). Según el comentario de ese archivo, el codo se limita artificialmente a la mitad de su rango ($\pm\pi$) para evitar problemas en planificadores como MoveIt/OMPL. En cada iteración la solución se recorta a estos límites con `numpy.clip`.

Cuadro IV
LÍMITES ARTICULARES UTILIZADOS (UR10E)

Articulación	Mínimo [rad]	Máximo [rad]
q_1 (shoulder_pan)	-6.2832	6.2832
q_2 (shoulder_lift)	-6.2832	6.2832
q_3 (elbow)	-3.1416	3.1416
q_4 (wrist_1)	-6.2832	6.2832
q_5 (wrist_2)	-6.2832	6.2832
q_6 (wrist_3)	-6.2832	6.2832

VI-G. Criterio de convergencia

El error euclidiano de posición es $e_p = \|p_d - p(q_k)\|_2$. El algoritmo termina cuando $e_p < \varepsilon = 10^{-4}$ m (convergencia) o al alcanzar 300 iteraciones (no convergencia), con $\alpha = 0.5$ y `paso_max` = 0.2 rad.

VI-H. Publicación y verificación de la solución

Si el algoritmo converge, q^* se publica en `/joint_states` (`sensor_msgs/msg/JointState`) con los nombres y el orden del URDF, lo que permite que `robot_state_publisher` actualice TF y la visualización en RViz2. Antes de publicarla, la solución se verifica con la cinemática directa propia, $p_f = p(q^*)$, comprobando que $\|p_d - p_f\|_2 < \varepsilon$. Si no converge, la solución no se publica y el robot conserva su pose anterior.

VII. IMPLEMENTACIÓN EN PYTHON Y ROS 2

VII-A. Herramientas

La implementación se desarrolló en Ubuntu 24.04 con ROS 2 Jazzy, Python 3.12 y NumPy. Se utilizaron los paquetes `robot_state_publisher`, `joint_state_publisher_gui`, `rviz2`, `xacro` y `tf2_ros`, con el middleware CycloneDDS (`rmw_cyclonedds_cpp`). SymPy se empleó únicamente para la derivación simbólica del informe (`scripts/dh_sympy.py`).

El código se organiza en el package `grupo06_ur10e_kinematics` (`ament_python`), con tres módulos:

- `ur10e_dh.py`: modelo cinemático sin dependencias de ROS. Contiene la tabla DH (Tabla II), la función que construye cada matriz A_i (ecuación 2), la cinemática directa $T_0^6 = A_1 A_2 \dots A_6$, el Jacobiano posicional geométrico $J_{v_i} = z_{i-1} \times (o_6 - o_{i-1})$ y la conversión de R_0^6 a cuaternión.
- `fk_node.py`: nodo de cinemática directa.
- `ik_node.py`: nodo de cinemática inversa.

Ambos nodos importan el mismo módulo `ur10e_dh.py`, de modo que la cinemática directa usada para resolver la IK es exactamente la misma que se valida contra TF.

VII-B. Nodo de cinemática directa (`fk_node.py`)

El nodo se suscribe a `/joint_states` (`sensor_msgs/msg/JointState`). Como el orden de las articulaciones en el mensaje no está garantizado, construye un diccionario `nombre → posición` y arma el vector $q = [q_1, \dots, q_6]$ en el orden del URDF (`shoulder_pan_joint`, ..., `wrist_3_joint`). Con ese vector evalúa numéricamente T_0^6 mediante las matrices DH propias y obtiene la posición $p = (x, y, z)$, la matriz R_0^6 y el cuaternión (x, y, z, w) .

Como referencia de validación, el nodo consulta en TF la transformación `base → tool0` publicada por `robot_state_publisher` y reporta el error de posición $\|p_{calc} - p_{ROS}\|_2$ y el error de orientación $2 \arccos |q_{calc} \cdot q_{ROS}|$. TF no interviene en el cálculo de la cinemática directa. Para no saturar la consola, los resultados se imprimen una vez por segundo y solo cuando q cambia.

VII-C. Nodo de cinemática inversa (`ik_node.py`)

El nodo se suscribe a `/target` (`geometry_msgs/msg/Point`). Al recibir un objetivo p_d , ejecuta el método iterativo de la Sección VI a partir de la semilla q_0 : calcula $p(q_k)$ con la FK propia, el error $e_k = p_d - p(q_k)$, el Jacobiano $J_v(q_k)$ de 3×6 y la actualización $q_{k+1} = q_k + \alpha J_k^+ e_k$, donde J_k^+ se obtiene con `numpy.linalg.pinv`. En cada iteración el paso se escala para que ninguna articulación cambie más de `paso_max`, y el resultado se recorta a los límites articulares (Tabla IV).

Los parámetros del algoritmo se declaran como parámetros de ROS 2 y pueden modificarse sin editar el código:

Para cada objetivo, el nodo reporta en consola el objetivo p_d , la semilla q_0 , la solución q^* , el número de iteraciones, la posición alcanzada $p(q^*)$ verificada con la cinemática directa, el error final y si hubo convergencia. Si converge, q^* pasa a ser el estado publicado; si no converge, el nodo emite una advertencia y el robot conserva su pose anterior.

El estado articular se publica en `/joint_states` a 10 Hz, con los nombres y el orden del URDF y con el sello de tiempo del reloj de ROS. La publicación es continua porque TF descarta las transformaciones antiguas: si el estado se publicara una sola vez, el modelo dejaría de mostrarse en RViz2.

El flujo de información es:

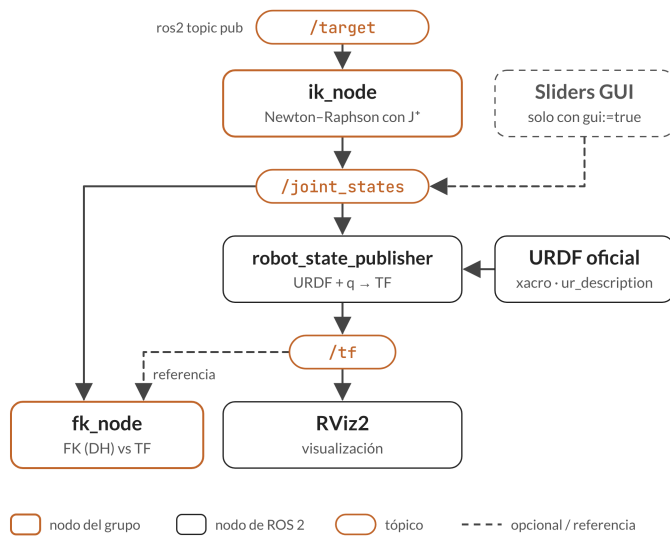


Figura 3. Arquitectura de nodos y tópicos. Con `gui:=false`, el `ik_node` es el único publicador de `/joint_states`.

VII-D. Interacción con Joint State Publisher GUI

El Joint State Publisher GUI también publica en `/joint_states`. Si este y el `ik_node` se ejecutan a la vez, `robot_state_publisher` recibe dos configuraciones distintas y el modelo alterna entre ambas. Para evitarlo, el launch del grupo (`display.launch.py`) incluye el argumento `gui`: con `gui:=true` (por defecto) se abre el GUI para mover el robot manualmente y probar la FK; con `gui:=false` no se lanza, y el `ik_node` es el único publicador de estados articulares. Se verificó con `ros2 topic info /joint_states` que en ese modo existe un único publicador.

VIII. WORKSPACE ROS 2

VIII-A. Estructura

El repositorio corresponde a la raíz del workspace. El paquete oficial `ur_description` no se versiona: se descarga con `vcstool` desde `dependencias.repos`, fijado al commit `6b639c2`. Los directorios `build/`, `install/` y `log/` se excluyen mediante `.gitignore`.

```
grupo06_ur10e_ws/
|-- src/
|   |-- grupo06_ur10e_bringup/                (ament_cmake
|   |)
|   |   |-- launch/display.launch.py
|   |   |-- rviz/ur10e.rviz
|   |   |-- CMakeLists.txt
|   |   |-- package.xml
|   |-- grupo06_ur10e_kinematics/            (
|   |   ament_python)
|   |   |-- grupo06_ur10e_kinematics/
|   |   |   |-- ur10e_dh.py
|   |   |   |-- fk_node.py
|   |   |   |-- ik_node.py
|   |   |-- test/test_ur10e_dh.py
|   |   |-- resource/
|   |   |-- package.xml
|   |   |-- setup.py
|   |   |-- setup.cfg
|   |-- ur_description/                      (descargado,
|   |   no versionado)
|-- scripts/dh_sympy.py
|-- docs/
|-- dependencias.repos
|-- instalar.sh
|-- entorno.sh
|-- README.md
|-- requirements.txt
-- .gitignore
```

VIII-B. Compilación

Desde la raíz del workspace (equivalente a ejecutar `./instalar.sh`):

```
source /opt/ros/jazzy/setup.bash
export RMW_IMPLEMENTATION=rmw_cyclonedds_cpp
vcs import src < dependencias.repos
rosdep install --from-paths src --ignore-src -r -y
colcon build --symlink-install
source install/setup.bash
```

En cada terminal nueva basta con ejecutar `source entorno.sh`.

VIII-C. Ejecución del robot

Con el GUI (pruebas de FK):

```
ros2 launch grupo06_ur10e_bringup display.launch.py
```

Sin el GUI (pruebas de IK):

```
ros2 launch grupo06_ur10e_bringup display.launch.py
gui:=false
```

VIII-D. Ejecución de los nodos propios

Cinemática directa:

```
ros2 run grupo06_ur10e_kinematics fk_node
```

Cinemática inversa:

```
ros2 run grupo06_ur10e_kinematics ik_node
```

Envío de un objetivo desde otra terminal:

```
ros2 topic pub --once /target geometry_msgs/msg/
Point \
"{x:_0.40,_y:_0.20,_z:_0.50}"
```

IX. VALIDACIÓN DE LA CINEMÁTICA DIRECTA

Como referencia se utilizó la transformación `base→tool0` que publica TF, calculada por `robot_state_publisher` a partir del URDF oficial de Universal Robots (`ur_description`, commit `6b639c2`). Esta referencia es independiente de nuestro modelo: proviene de la descripción del fabricante y no de las matrices DH del grupo.

Para los casos 2 a 4, el estado articular se publicó con valores exactos mediante `ros2 topic pub` (con el GUI cerrado), de modo que las pruebas son reproducibles, y la comparación la realizó automáticamente el `fk_node` con precisión completa. El caso 1 se obtuvo moviendo los sliders del Joint State Publisher GUI y la referencia se leyó con `ros2 run tf2_ros tf2_echo base tool0`, que muestra solo tres decimales; por eso su error refleja ese redondeo y no un error del modelo.

Los errores se calcularon como:

$$e_p = \|p_{\text{calc}} - p_{\text{ROS}}\|_2, \quad e_o = 2 \arccos|\mathbf{q}_{\text{calc}} \cdot \mathbf{q}_{\text{ROS}}|, \quad (16)$$

donde $\mathbf{q}$ es el cuaternión de orientación; se usa el valor absoluto porque $\mathbf{q}$ y $-\mathbf{q}$ representan la misma rotación.

En los cuatro casos, todos dentro de los límites articulares y con al menos una configuración con ángulos grandes en las seis articulaciones (caso 4), la cinemática directa propia coincide con la del modelo oficial por debajo de 10^{-6} m y 10^{-6} rad, es decir, al nivel del redondeo numérico. Como comprobación adicional, en $q = 0$ se obtuvo $p = [-1.1843, -0.2907, 0.0609]$ m, que se lee directamente de la tabla DH: $x = a_2 + a_3$, $y = -(d_4 + d_6)$ y $z = d_1 - d_5$.

IX-A. Pruebas por transformaciones parciales

La cadena completa T_0^6 coincidió con TF desde la primera evaluación con la tabla final, por lo que no fue necesario aislar errores eslabón por eslabón. Cabe señalar que los marcos intermedios DH no coinciden con los marcos de los eslabones del URDF (por ejemplo, los orígenes O_1 , O_2 y O_3 quedan en el plano central del robot y los desplazamientos laterales se acumulan en d_4), de modo que la comparación directa con TF solo es válida para los marcos extremos `base` y `tool0`. Durante el desarrollo se evaluaron dos asignaciones de marcos distintas (sin offsets y con offsets de π en θ_1 y θ_3); ambas reprodujeron la misma T_0^6 , lo que confirma que la tabla DH no es única aunque la cinemática sí lo es. La ortogonalidad de la rotación se verificó en todos los casos: $(R_0^6)^T R_0^6 = I$ y $\det(R_0^6) = 1$ con error del orden de 10^{-16} .

X. VALIDACIÓN DE LA CINEMÁTICA INVERSA

Los objetivos se enviaron con `ros2 topic pub -once /target geometry_msgs/msg/Point` y el robot se lanzó con `gui:=false`, de modo que el `ik_node` fue el único publicador de `/joint_states`. En todos los casos se partió de la misma semilla $q_0 = [0, -\frac{\pi}{2}, \frac{\pi}{2}, -\frac{\pi}{2}, -\frac{\pi}{2}, 0]$ rad, con $\alpha = 0.5$, $\varepsilon = 10^{-4}$ m, `paso_max = 0.2` rad y un máximo de 300 iteraciones. Cada solución se verificó además con el `fk_node` contra TF, con error menor que 10^{-6} m en los casos convergentes.

Los cuatro objetivos alcanzables convergieron en 12 a 46 iteraciones con error final menor que la tolerancia. El caso 4, que es el ejemplo de la plantilla, requirió más iteraciones porque el objetivo se encuentra detrás de la base respecto de la semilla: la solución gira $q_1 \approx -2.82$ rad y el límite de paso hace que ese giro sea gradual. El caso 5 se encuentra a aproximadamente 1.5 m del hombro, fuera del alcance del UR10e (unos 1.3 m); el algoritmo extiende el brazo hasta quedar a 0.143 m del objetivo, cerca de la singularidad de codo ($q_3 \approx 0.16$ rad), y el nodo lo reporta como no convergente.

En todas las soluciones q_6 conserva el valor de la semilla: la columna 6 de J_v es nula porque el origen de `tool0` está sobre el eje z_5 , y la pseudoinversa, al entregar la solución de norma mínima, no asigna movimiento a una articulación que no modifica la posición.

XI. REPOSITORIO GITHUB Y REPRODUCIBILIDAD

XI-A. Enlace

- Repositorio: https://github.com/chicho022/grupo_06_ur10e_ws
- Rama de entrega: `main`
- Commit evaluado: `7be7932ae0fc35a3f650d9c0e3035e321663679b`

XI-B. Contenido del repositorio

El repositorio contiene el código fuente de los paquetes `grupo06_ur10e_bringup` (launch y configuración de RViz2) y `grupo06_ur10e_kinematics` (modelo DH y nodos de FK e IK), sus archivos `package.xml`, `setup.py` y `CMakeLists.txt`, el script de derivación simbólica `scripts/dh_sympy.py`, los archivos `dependencias.repos`, `instalar.sh`, `entorno.sh`, `requirements.txt`, `.gitignore` y `README.md`, y la carpeta `docs/` con este informe, los registros de validación (`validacion_fk.md`, `validacion_ik.md`) y las capturas de RViz2.

No se versionan `build/`, `install/`, `log/`, `__pycache__/`, archivos `*.pyc`, carpetas de editores ni el paquete externo `src/ur_description/`.

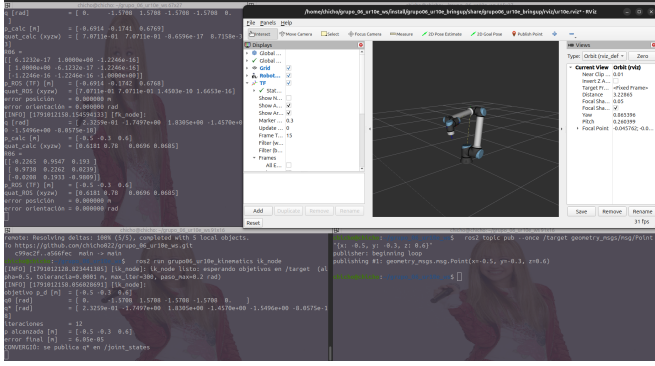
XI-C. Dependencias externas

El paquete oficial del robot se declara en `dependencias.repos`, fijado a un commit conocido para que una actualización futura no altere el proyecto:

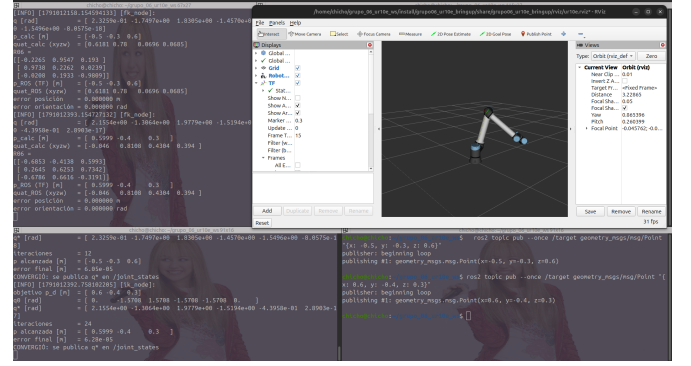
```
repositories:
  ur_description:
    type: git
    url: https://github.com/UniversalRobots/
    Universal_Robots_ROS2_Description.git
    version: 6
    b639c2efd9f4f12da858a72cf1a9e40da365ed8
```

XI-D. Procedimiento de reproducción

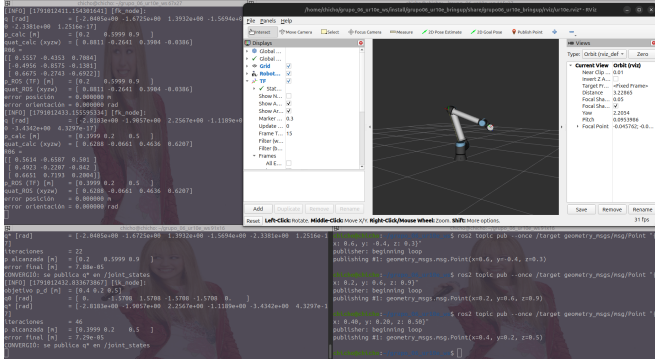
En una computadora con Ubuntu 24.04 y ROS 2 Jazzy:



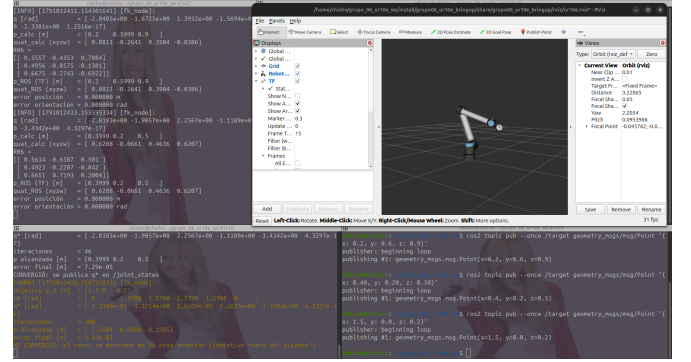
(a) Caso 1



(b) Caso 2



(c) Caso 4 (ejemplo de la plantilla)



(d) Caso 5: no convergencia

Figura 4. Validación de objetivos de cinemática inversa en RViz2. En cada captura se observa el robot en la solución q^* y el reporte del `ik_node`.

```
git clone https://github.com/chicho022/
  grupo_06_ur10e_ws.git
cd grupo_06_ur10e_ws
chmod +x instalar.sh
./instalar.sh
source entorno.sh
ros2 launch grupo06_ur10e_bringup display.launch.py
  gui:=false
```

y en otras terminales (cada una con `source entorno.sh`):

```
ros2 run grupo06_ur10e_kinematics ik_node
ros2 run grupo06_ur10e_kinematics fk_node
ros2 topic pub --once /target geometry_msgs/msg/
  Point "{x:_0.40,y:_0.20,z:_0.50}"
```

El script `instalar.sh` instala las dependencias del sistema, inicializa `rosdep`, descarga `ur_description` con `vcs import`, instala las dependencias declaradas en los `package.xml` y compila con `colcon build --symlink-install`. `entorno.sh` carga ROS 2, selecciona `CycloneDDS` y carga `install/setup.bash` desde la ruta donde se haya clonado el repositorio. El `README.md` describe además los tópicos y parámetros, la estructura del repositorio y los errores conocidos. El procedimiento se verificó clonando el repositorio en una segunda computadora.

XII. RESULTADOS Y DISCUSIÓN

Modelo frente a ROS 2. La cinemática directa obtenida con la tabla DH reprodujo la transformación `base→tool0` del URDF oficial con errores menores que 10^{-6} m y 10^{-6} rad en todas las configuraciones evaluadas, es decir, al nivel del redondeo numérico. El único error apreciable (0.1 mm)

corresponde al caso leído con `tf2_echo`, que muestra tres decimales. El Jacobiano geométrico coincidió con el analítico obtenido en SymPy (diferencia del orden de 10^{-17}) y con la derivada numérica de la FK.

Asignación de marcos. La principal dificultad fue que los marcos DH no coinciden con los marcos de los eslabones del URDF. El marco base DH corresponde a base y no a `base_link` (rotación fija de π), y los desplazamientos laterales que el URDF reparte entre varios eslabones se concentran en d_4 . También se comprobó que la tabla DH no es única: una asignación con x_1 y x_2 orientados hacia el codo (offsets de π en θ_1 y θ_3 , $\alpha_1 = -\pi/2$, $a_2 > 0$) produce exactamente la misma T_0^6 y el mismo J_v ; se eligió la tabla sin offsets por coincidir con la documentación del fabricante.

Cinemática inversa. Los cuatro objetivos alcanzables convergieron en 12 a 46 iteraciones. El número de iteraciones depende de la distancia entre la semilla y la solución: el ejemplo de la plantilla, ubicado detrás de la base, exigió girar q_1 casi -161° y requirió 46 iteraciones. Al ejecutar el mismo objetivo con $\alpha = 1$ y sin límite de paso (`-ros-args -p alpha:=1.0 -p paso_max:=100.0`), el método no convergió tras 300 iteraciones: los pasos excesivos llevaron q_1 y q_4 hasta su límite de 2π , donde quedaron saturadas, y el error final fue de 0.261 m pese a que el objetivo es alcanzable. Reducir α a 0.5 y limitar el paso a 0.2 rad mantiene cada actualización dentro de la zona donde la linealización $\Delta p \approx J_v \Delta q$ es válida, y el método convergió en todos los casos alcanzables. La solución encontrada depende de la semilla, ya que la IK de posición de un robot de 6 GDL tiene infinitas soluciones; además, q_6 nunca cambia porque

no influye en la posición.

Singularidades y alcance. Con el objetivo fuera del alcance (1.5 m), el algoritmo estiró el brazo hasta $q_3 \approx 0.16$ rad, cerca de la singularidad de codo, y se detuvo a 0.143 m del objetivo; el nodo lo reportó como no convergente sin mover el robot.

Integración en ROS 2. Se identificaron tres problemas prácticos: dos publicadores simultáneos en `/joint_states` (GUI e `ik_node`) hacen que el modelo alterne entre poses, por lo que se agregó el argumento `gui` al `launch`; dos computadoras en la misma red con el mismo `ROS_DOMAIN_ID` intercambian mensajes entre sí; y los mensajes `JointState` con sello de tiempo nulo impiden que `robot_state_publisher` publique TF, por lo que el `ik_node` usa el reloj de ROS y repubblica a 10 Hz.

XIII. CONCLUSIONES

- **Modelado cinemático.** La asignación de marcos y la tabla Denavit–Hartenberg fueron la base de todo el proyecto: a partir de ellas se obtuvieron la posición $p(q)$ y la orientación R_0^6 del efector final. La cinemática directa propia reprodujo la transformación `base→tool0` del URDF oficial con errores menores que 10^{-6} m y 10^{-6} rad. Además, se comprobó que la tabla DH no es única, aunque la cinemática resultante sí lo es, y que el marco base del modelo DH (`base`) no coincide con el de ROS (`base_link`), por lo que la rotación fija entre ambos debe considerarse al interpretar los resultados.
- **Relación entre cinemática directa e inversa.** Ambos problemas están directamente ligados: la cinemática inversa numérica usa la directa en cada iteración para medir el error, y la directa sirve después para verificar la solución. El Jacobiano posicional cumple el papel de la derivada en el método de Newton–Raphson: traduce el error cartesiano en una corrección articular, con lo que la configuración se ajusta iteración a iteración hasta alcanzar el punto deseado. Su sexta columna es nula, por lo que q_6 no interviene en la IK de posición.
- **Comportamiento del método numérico.** Con $\alpha = 0.5$ y un paso máximo de 0.2 rad, el método convergió en 12 a 46 iteraciones para todos los objetivos alcanzables, mientras que con $\alpha = 1$ y sin límite de paso no convergió, porque los pasos grandes salen de la zona donde la linealización $\Delta p \approx J_v \Delta q$ es válida. Fuera del espacio de trabajo, el nodo detectó correctamente la no convergencia y no publicó la solución.
- **Comunicación en ROS 2.** La práctica permitió aplicar los conceptos de nodo, tópico, publicador y suscriptor: el objetivo llega por `/target`, la solución se publica en `/joint_states` y `robot_state_publisher` la convierte en TF para su visualización en RViz2. Conocer el tópico y el tipo de mensaje de cada nodo resultó esencial, y fue también la clave para diagnosticar los problemas de integración, ninguno de los cuales generaba un error visible: dos publicadores en el mismo tópico (resuelto con `gui:=false`), nodos de otra computadora en el mismo dominio DDS y mensajes con sello de tiempo nulo.

- **Reproducibilidad y extensión.** Separar el modelo matemático de los nodos, fijar la dependencia oficial a un commit y automatizar la instalación permitió reproducir el proyecto desde cero en una segunda computadora. Esta misma estructura puede adaptarse a otro manipulador serial cambiando su descripción URDF y su tabla DH, sin modificar la lógica de los nodos.

REFERENCIAS

- [1] Universal Robots, “UR10e Technical Specifications,” [En línea]. Disponible: <https://www.universal-robots.com/products/ur10e/>. Consultado: oct. 2026.
- [2] Universal Robots, “DH parameters for calculations of kinematics and dynamics,” [En línea]. Disponible: <https://www.universal-robots.com/articles/ur/application-installation/dh-parameters-for-calculations-of-kinematics-and-dynamics/>. Consultado: oct. 2026.
- [3] Universal Robots, “Universal_Robots_ROS2_Description,” GitHub, rama `jazzy`, commit `6b639c2`. [En línea]. Disponible: https://github.com/UniversalRobots/Universal_Robots_ROS2_Description.
- [4] M. W. Spong, S. Hutchinson y M. Vidyasagar, *Robot Modeling and Control*, 2.^a ed. Hoboken, NJ, EE. UU.: Wiley, 2020.
- [5] Open Robotics, “ROS 2 Documentation: Jazzy,” [En línea]. Disponible: <https://docs.ros.org/en/jazzy/>. Consultado: oct. 2026.

Cuadro V
PARÁMETROS DEL NODO `IK_NODE`

Parámetro	Valor	Descripción
alpha	0.5	Factor de actualización
tolerancia	10^{-4} m	Criterio $\ e_k\ _2 < \varepsilon$
max_iter	300	Máximo de iteraciones
paso_max	0.2 rad	Cambio máx. por articulación
q0	$[0, -\frac{\pi}{2}, \frac{\pi}{2}, -\frac{\pi}{2}, -\frac{\pi}{2}, 0]$	Semilla

Cuadro VI
VALIDACIÓN DE LA CINEMÁTICA DIRECTA

Caso	q [rad]	p_{calc} [m]	p_{ROS} [m]	q_{calc} (x, y, z, w)	e_p [m]	e_o [rad]
1	$[-4.2736, -1.0610, -1.6986, -4.8060, -1.3640, 0.4428]$	$[0.1160, 0.2190, 0.7851]$	$[0.116, 0.219, 0.785]^*$	$[0.0168, 0.9843, 0.1728, 0.0314]$	$1.0 \times 10^{-4} *$	$\approx 10^{-3} *$
2	$[0.5, -1.2, 1.0, -1.4, -1.57, 0.3]$	$[-0.7110, -0.5870, 0.7523]$	$[-0.7110, -0.5870, 0.7523]$	$[0.6329, 0.7741, -0.0137, -0.0051]$	$< 10^{-6}$	$< 10^{-6}$
3	$[-1.0, -0.8, -1.5, 0.7, 1.2, -0.5]$	$[-0.2700, 0.0200, 1.1585]$	$[-0.2700, 0.0200, 1.1585]$	$[-0.1512, -0.1064, 0.9822, -0.0321]$	$< 10^{-6}$	$< 10^{-6}$
4	$[2.0, -2.0, 2.2, -2.5, 0.6, 1.5]$	$[0.3918, -0.2064, 0.7532]$	$[0.3918, -0.2064, 0.7532]$	$[-0.0489, 0.5358, 0.5843, 0.6075]$	$< 10^{-6}$	$< 10^{-6}$

*Referencia leída con `tf2_echo` (3 decimales); el error corresponde al redondeo de la referencia.

Cuadro VII
PRUEBAS DE CINEMÁTICA INVERSA (q_0 IGUAL EN TODOS LOS CASOS)

Caso	$p_d = [x_d, y_d, z_d]$ [m]	q^* [rad]	$p(q^*)$ [m]	Iter.	Error final [m]	Converge
1	$[-0.50, -0.30, 0.60]$	$[0.2326, -1.7497, 1.8305, -1.4570, -1.5496, 0]$	$[-0.5000, -0.3000, 0.6000]$	12	6.05×10^{-5}	Sí
2	$[0.60, -0.40, 0.30]$	$[2.1554, -1.3064, 1.9779, -1.5194, -0.4396, 0]$	$[0.5999, -0.4000, 0.3000]$	24	6.28×10^{-5}	Sí
3	$[0.20, 0.60, 0.90]$	$[-2.0405, -1.6725, 1.3932, -1.5694, -2.3381, 0]$	$[0.2000, 0.5999, 0.9000]$	22	7.88×10^{-5}	Sí
4	$[0.40, 0.20, 0.50]$	$[-2.8183, -1.9057, 2.2567, -1.1189, -3.4342, 0]$	$[0.3999, 0.2000, 0.5000]$	46	7.29×10^{-5}	Sí
5	$[1.50, 0.00, 0.20]$	$[0.1320, -3.2254, 0.1621, -2.2623, -1.3703, 0]^\dagger$	$[1.3584, -0.0188, 0.1985]$	300	1.43×10^{-1}	No

† Mejor aproximación tras 300 iteraciones; no se publica y el robot conserva su pose anterior.